

ClusterIP Service

```
apiVersion: v1
kind: Service
metadata:
  name: demo-service
spec:
  selector:
    app: demo-app
  ports:
    - port: 80

    targetPort: 8080
```

NodePort Service

```
apiVersion: v1
kind: Service
metadata:
  name: demo-service
spec:
  type: NodePort
  selector:
    app: demo-app
  ports:
    - port: 80

    targetPort: 8080

    nodePort: 30090
```

LoadBalancer Service

```
apiVersion: v1
kind: Service
metadata:
  name: demo-service
spec:
  type: LoadBalancer
  selector:
    app: demo-app
```

ports:
- port: 80
targetPort: 8080

ExternalName Service

apiVersion: v1
kind: Service
metadata:
 name: demo-ext-service
spec:
 type: ExternalName
 externalName: db.example.com

Discovering services across namespaces

curl http://service1

curl <http://service1.namespace1>

curl <http://service1.namespace1.svc.cluster.local>

Headless Services

apiVersion: v1
kind: Service
metadata:
 name: demo-db
 namespace: database
spec:
 ClusterIP: None
 selector:
 app: demo-db
 ports:
 - port: 5432
 targetPort: 5432

Basic Ingress Configuration

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: basic-ingress
  annotations:
    nginx.ingress.kubernetes.io/rewrite-target: /
spec:
  rules:
  - host: demo.example.com
    http:
      paths:
      - path: /web1
        pathType: Prefix
        backend:
          service:
            name: web1-service
            port:
              number: 80
      - path: /web2
        pathType: Prefix
        backend:
          service:
            name: web2-service
            port:
              number: 80
```

TLS Termination and HTTPS

```
apiVersion: v1
kind: Secret
metadata:
  name: secret-tls
  namespace: default
data:
  tls.crt: base64 encoded cert
  tls.key: base64 encoded key
```

type: kubernetes.io/tls

apiVersion: networking.k8s.io/v1

kind: Ingress

metadata:

name: tls-ingress

spec:

tls:

- hosts:

- https-demo.example.com

secretName: secret-tls

rules:

- host: https-demo.example.com

http:

paths:

- path: /

pathType: Prefix

backend:

service:

name: service1

port:

number: 80

Advanced Ingress features

rules:

- host: app.example.com

http:

paths:

- path: /api

pathType: Prefix

backend:

service:

name: api-service

port:

number: 80

- path: /web

pathType: Prefix

backend:

service:

name: web-service

port:
number: 80

annotations:
nginx.ingress.kubernetes.io/rewrite-target: /
nginx.ingress.kubernetes.io/ssl-redirect: "true"
nginx.ingress.kubernetes.io/force-ssl-redirect: "true"

annotations:
nginx.ingress.kubernetes.io/limit-connections: "20"
nginx.ingress.kubernetes.io/limit-rpm: "60"

annotations:
nginx.ingress.kubernetes.io/auth-type: basic
nginx.ingress.kubernetes.io/auth-secret: basic-auth
nginx.ingress.kubernetes.io/auth-realm: "Authentication Required"

annotations:
nginx.ingress.kubernetes.io/canary: "true"
nginx.ingress.kubernetes.io/canary-weight: "20"

spec:
ingressClassName: nginx

annotations:
nginx.ingress.kubernetes.io/custom-http-errors: "404,502"
nginx.ingress.kubernetes.io/configuration-snippet: |
more_set_headers "Request-Id: \$req_id";

Creating Network Policies

apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:

```
  name: default-deny-ingress
spec:
  podSelector: {}
  policyTypes:
  - Ingress
```

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-all-ingress
spec:
  podSelector: {}
  ingress:
  - {}
  policyTypes:
  - Ingress
```

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: default-deny-egress
spec:
  podSelector: {}
  policyTypes:
  - Egress
```

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-all-egress
spec:
  podSelector: {}
  egress:
  - {}
  policyTypes:
  - Egress
```

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
```

```
metadata:
  name: default-deny-all
spec:
  podSelector: {}
  policyTypes:
  - Ingress
  - Egress
```

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-frontend-to-backend
spec:
  podSelector:
    matchLabels:
      app: backend
  policyTypes:
  - Ingress
  ingress:
  - from:
    - podSelector:
        matchLabels:
          app: frontend
```

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-egress-to-internet
spec:
  podSelector:
    matchLabels:
      app: ext-client
  policyTypes:
  - Egress
  egress:
  - to:
    - ipBlock:
        cidr: 0.0.0.0/0
  ports:
  - protocol: TCP
    port: 443
```

